



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

**FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA INGENIERÍA DE SOFTWARE**

Tema:

Temario de preguntas

Integrantes:

Díaz Pontón Steven Santiago
Gamarra Zárate Jamileth Estefania
Herrera Ramos Thais Melanie
Tigasi Sampedor Paúl Alexander
Trujillo Vega Mayummy Jaily

Curso/Paralelo:

4to "A"

Docente:

Ing. Portilla Olvera Elías

Repaso general de la materia basado en preguntas y consignas.

En esta actividad, deberá realizar un repaso guiado por temas examinados en el primer parcial en la asignatura de Sistemas Operativos, que debe revisar a modo de preparación para la evaluación parcial de la semana 9, contestando las siguientes preguntas y resolviendo las consignas que se indican:

PARCIAL 1

1. Indique cuáles son las funciones principales de un sistema operativo tradicional como Windows 10 o Ubuntu Desktop.

Las funciones principales de un sistema operativo tradicional incluyen:

- **Gestión de procesos:** administrar la creación, ejecución y finalización de procesos y, en sistemas como Windows, hilos. Incluye la planificación para asignar tiempo de CPU.
- **Gestión de memoria:** administrar la memoria principal (RAM), subdividirla para alojar múltiples procesos y traducir direcciones lógicas a físicas, mediante técnicas como paginación o segmentación.
- **Gestión de archivos:** proporcionar un sistema de archivos para organizar, almacenar y recuperar datos en dispositivos de almacenamiento secundario (discos duros, SSD).
- **Gestión de entrada/salida (E/S):** controlar los dispositivos de E/S, proporcionando una interfaz uniforme y manejando las transferencias de datos, por ejemplo mediante buffers.
- **Seguridad y protección:** implementar mecanismos para proteger los recursos del sistema y la información de los usuarios, mediante dominios de protección, permisos de acceso y autenticación.
- **Interfaz de usuario:** proporcionar una manera para que los usuarios interactúen con el sistema, mediante consola (CLI) o interfaz gráfica (GUI).

2. Describa fragmentación externa.

La fragmentación externa es un problema que ocurre principalmente en los esquemas de particionamiento dinámico de memoria. A medida que los procesos entran y salen de la memoria, se crean múltiples bloques de memoria libre de diferentes tamaños que se intercalan entre los bloques ocupados por los procesos.

Aunque la suma total de memoria libre podría ser suficiente para un nuevo proceso, no existe un solo bloque contiguo lo bastante grande para alojarlo. Esta memoria libre queda “fragmentada” y sin poder ser utilizada. Una técnica para superar esto es la compactación, que reorganiza los procesos para que ocupen un bloque contiguo y la memoria libre se una en un solo bloque.

3. Defina fragmentación interna.

La fragmentación interna es el espacio de memoria desperdiciado dentro de una partición o marco de página asignado a un proceso. Ocurre cuando el tamaño de la partición o marco es mayor que el espacio real que necesita el proceso que la ocupa, dejando una porción de memoria que no puede ser utilizada.

Es típica del particionamiento fijo y de la paginación. Por ejemplo, en un sistema con particiones fijas de 8 MB, un proceso que solo necesita 2 MB ocupará la partición completa, desperdiciando 6 MB. En la paginación, el desperdicio se limita al espacio no utilizado en la última página del proceso.

4. Explique el esquema de paginación que se utiliza en la gestión de memoria.

La paginación es una técnica de gestión de memoria que elimina la fragmentación externa. Su funcionamiento se basa en lo siguiente:

- La memoria principal (RAM) se divide en bloques de tamaño fijo llamados marcos.
- Cada proceso se divide en bloques del mismo tamaño fijo llamados páginas.
- Las páginas de un proceso se cargan en marcos disponibles, sin necesidad de que sean contiguos.
- El sistema operativo mantiene una tabla de páginas por proceso, que traduce la dirección virtual (número de página y desplazamiento) a la dirección física (número de marco y desplazamiento).

Esta técnica evita la fragmentación externa porque las páginas, al ser pequeñas y de tamaño fijo, encajan en cualquier marco libre. La única fragmentación que persiste es la interna en el último marco ocupado por el proceso.

5. Narre el esquema de segmentación que se utiliza en la gestión de memoria.

La segmentación divide el espacio de direcciones de un proceso en segmentos de tamaño variable. Cada segmento representa una parte lógica del programa (código, datos, pila o una biblioteca).

El sistema operativo mantiene una tabla de segmentos por proceso, que registra para cada segmento su dirección base en memoria física y su longitud (límite). Las principales ventajas son:

- **Organización lógica:** permite una estructura más cercana a la forma en que se construyen los programas.
- **Compartición:** facilita compartir código o datos entre procesos a nivel de segmento.
- **Protección:** permite asignar distintos permisos (lectura, escritura, ejecución) a cada segmento.

Su principal desventaja es que, igual que el particionamiento dinámico, produce fragmentación externa debido a la naturaleza variable de los segmentos.

6. Liste las condiciones que se deben cumplir para que exista la probabilidad de un interbloqueo sin que este llegue a ocurrir.

Las cuatro condiciones de Coffman son necesarias para que ocurra un interbloqueo. Si están presentes, existe la posibilidad, pero no la certeza, de que ocurra:

- **Exclusión mutua:** al menos un recurso debe ser no compartible, es decir, solo un proceso puede usarlo a la vez.
- **Retener y esperar:** un proceso retiene al menos un recurso mientras espera por otros recursos retenidos por otros procesos.
- **No apropiación:** los recursos no pueden ser arrebatados a los procesos que los retienen; deben liberarse voluntariamente.
- **Espera circular:** existe una cadena de dos o más procesos donde cada uno espera un recurso retenido por el siguiente de la cadena.

7. Liste las condiciones que se deben cumplir para que ocurra un interbloqueo.

Para que un interbloqueo ocurra definitivamente, deben cumplirse simultáneamente las cuatro condiciones de Coffman descritas en la pregunta anterior: exclusión mutua, retener y esperar, no apropiación, y espera circular. Si

alguna no se cumple, el interbloqueo no puede ocurrir; por ello, las estrategias de prevención o evitación de interbloques buscan asegurar que al menos una de estas condiciones nunca se cumpla.

8. Defina proceso.

Un proceso se define como un programa en ejecución. Es la entidad que se puede asignar para ser ejecutada en un procesador. Un proceso es más que el código del programa; también incluye:

- Su estado actual de ejecución (ejecutando, listo, bloqueado, etc.).
- Un espacio de direcciones virtuales.
- Recursos del sistema asociados (archivos abiertos, dispositivos de E/S).
- Su contexto de ejecución (contenido de los registros del procesador cuando no está en ejecución).

9. Defina el concepto de interrupción.

Una interrupción es una señal que recibe el procesador que altera la secuencia normal de ejecución de instrucciones. Es un mecanismo fundamental que permite al sistema operativo recuperar el control del procesador cuando ocurre un evento importante.

Las interrupciones pueden ser generadas por hardware (por ejemplo, la finalización de una operación de E/S o un error de división por cero) o por software (a través de llamadas al sistema, también llamadas interrupciones de software). Cuando ocurre una interrupción, el procesador detiene la ejecución del proceso actual, guarda su contexto y transfiere el control a una rutina específica del sistema operativo (el manejador de interrupciones) para atender el evento.

10. Describa el concepto de Sistema Operativo.

Un Sistema Operativo (SO) es un software fundamental que actúa como interfaz entre el hardware de la computadora y el usuario (o las aplicaciones). Sus principales funciones son:

- Gestionar los recursos del hardware (procesador, memoria, dispositivos de E/S) de manera eficiente.
- Proporcionar un entorno en el que los programas de aplicación puedan ejecutarse.
- Crear abstracciones del hardware para simplificar la programación (por ejemplo, el concepto de archivo).
- Garantizar la seguridad y protección del sistema y los datos de los usuarios.

Se puede ver como un conjunto de capas que van desde el hardware en el nivel más bajo hasta la interfaz de usuario en el nivel más alto.

11. Puntualice el concepto de Sistema Operativo de tiempo real. Dé un ejemplo.

Un sistema operativo de tiempo real (RTOS) es un sistema operativo diseñado para proporcionar respuestas predecibles y deterministas, priorizando la capacidad de responder a eventos dentro de plazos (deadlines) establecidos sobre el rendimiento medio.

No basta con que una operación sea lógicamente correcta; también debe completarse en el momento exacto requerido. Si una tarea no se completa a tiempo, se considera un fallo, incluso si el resultado es correcto.

- **Ejemplo de tiempo real duro:** el sistema de control de un marcapasos o el sistema de frenos ABS de un automóvil, donde un fallo en el tiempo de respuesta puede tener consecuencias fatales.

- **Ejemplo de tiempo real blando:** un sistema de streaming de video, donde un pequeño retraso puede causar una pausa molesta, pero no es catastrófico.

12. Explique y grafique el funcionamiento de la memoria caché en un solo nivel.

La memoria caché es una memoria de alta velocidad y pequeña capacidad ubicada entre la CPU y la memoria principal (RAM). Su funcionamiento se basa en el principio de localidad (temporal y espacial) para acelerar el acceso a los datos e instrucciones más utilizados.

- Cuando la CPU necesita un dato (DL), primero busca el bloque que lo contiene en la caché L1.
- Si el dato está en la caché (acierto o hit), se entrega a la CPU a gran velocidad.
- Si el dato no está en la caché (fallo o miss), la CPU espera mientras el controlador de caché accede a la memoria principal para buscar el bloque que contiene DL.
- Ese bloque se carga en la caché, generalmente reemplazando otro bloque, y luego se entrega a la CPU.

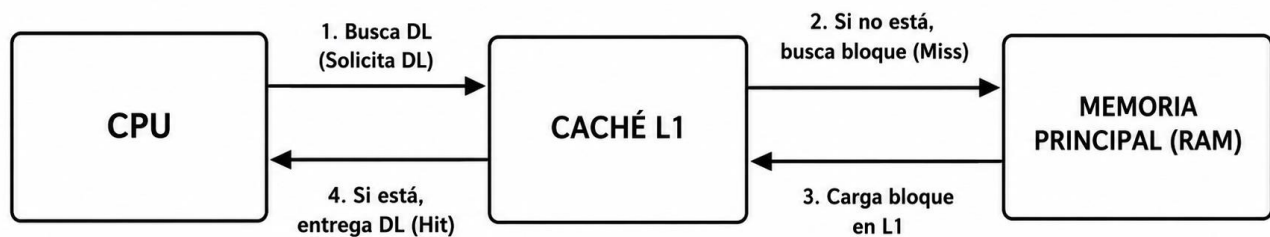


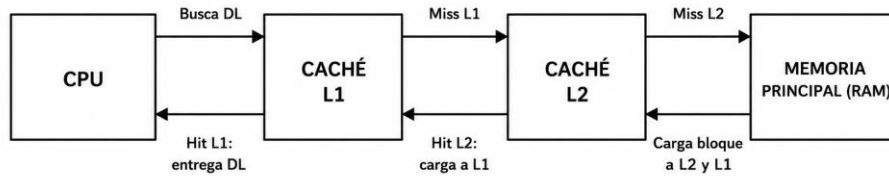
Figura 1. Funcionamiento de la memoria caché de un solo nivel.

13. Explique y grafique el funcionamiento de la memoria caché en dos niveles.

La caché de dos niveles (L1 y L2) añade una capa intermedia para mejorar aún más el rendimiento. La L1 es la más rápida y pequeña; la L2 es más grande pero ligeramente más lenta.

- La CPU busca el bloque que contiene el dato DL en la caché L1.
- Si está en L1 (acierto), se entrega el dato a la CPU y el proceso termina.
- Si no está en L1 (fallo), se busca el bloque en la caché L2.
- Si está en L2 (acierto), el bloque se carga en L1 para futuros accesos y luego se entrega DL a la CPU.
- Si no está en L2 (fallo), se accede a la memoria principal (RAM) para buscar el bloque.
- El bloque se carga en L2, luego en L1 y finalmente se entrega DL a la CPU.

Funcionamiento de la memoria caché — dos niveles (L1 y L2)



Si falla en L1 y L2, se accede a RAM; el bloque se propaga hacia L2 y luego hacia L1

Figura 2. Funcionamiento de la memoria caché de dos niveles (L1 y L2).

14. Describa el concepto de memoria caché.

La memoria caché es un componente de hardware de alta velocidad que actúa como un buffer entre el procesador y la memoria principal (RAM). Su propósito es reducir el tiempo de acceso a los datos e instrucciones que la CPU necesita, basándose en la localidad de referencias.

Debido a que la CPU es mucho más rápida que la RAM, la memoria caché almacena copias de los bloques de datos que se utilizan con frecuencia. Cuando la CPU solicita un dato, primero se busca en la caché: si está allí, la operación es muy rápida; si no, se debe acceder a la RAM, lo cual es mucho más lento.

15. ¿Qué es un sistema embebido? Dé un ejemplo.

Un sistema empujado (o embebido) es un sistema computacional de uso específico, diseñado para realizar una función o un conjunto reducido de funciones dentro de un dispositivo más grande. A diferencia de una computadora de propósito general, combina hardware y software dedicados para cumplir una tarea concreta de manera eficiente y confiable. A menudo tiene recursos limitados (poca memoria y baja potencia de procesamiento) y, en muchos casos, debe cumplir requisitos de tiempo real.

Ejemplo: el sistema de control de un horno microondas (controla el tiempo y la potencia), un marcapasos, la unidad de control del motor (ECU) de un automóvil, el firmware de un router o los sistemas de una lavadora.

16. Suponga que en un ordenador se dispone de 2GB de memoria RAM y se gestiona la memoria mediante paginación. Los procesos ocupan páginas de 5 MB y ocupan marcos de páginas de 5MB. Existen 4 procesos en ejecución y cada uno de ellos ocupa respectivamente 35 MB, 36 MB, 28 MB y 25 MB. Realice los siguientes cálculos:

Proceso	Tamaño real	Páginas necesarias	Memoria asignada	Frag. interna
Proceso 1	35 MB	$\lceil 35/5 \rceil = 7$	$7 \times 5 = 35$ MB	0 MB
Proceso 2	36 MB	$\lceil 36/5 \rceil = 8$	$8 \times 5 = 40$ MB	4 MB
Proceso 3	28 MB	$\lceil 28/5 \rceil = 6$	$6 \times 5 = 30$ MB	2 MB
Proceso 4	25 MB	$\lceil 25/5 \rceil = 5$	$5 \times 5 = 25$ MB	0 MB

a) **Fragmentación interna total:** $0 + 4 + 2 + 0 = 6$ MB.

b) Fragmentación externa: 0 MB. En un sistema de paginación pura no existe fragmentación externa, ya que las páginas, al ser de tamaño fijo, pueden ocupar cualquier marco libre; la memoria libre queda repartida en marcos, pero todos son del mismo tamaño y pueden asignarse a cualquier página.

c) Cantidad de marcos de páginas utilizados: $7 + 8 + 6 + 5 = 26$ marcos.

d) Memoria disponible utilizable: la memoria total es $2 \text{ GB} = 2 \times 1024 = 2048 \text{ MB}$. La memoria asignada (utilizable) a los procesos es $35 + 40 + 30 + 25 = 130 \text{ MB}$, lo que equivale a $26 \text{ marcos} \times 5 \text{ MB} = 130 \text{ MB}$.

17. Indique al menos cinco estados de los procesos.

- **Nuevo:** el proceso se está creando.
- **Listo:** el proceso está en memoria y esperando a ser asignado al procesador para ejecutarse.
- **Ejecutando:** las instrucciones del proceso están siendo ejecutadas activamente por el procesador.
- **Bloqueado (o en espera):** el proceso espera la ocurrencia de algún evento (como la finalización de una operación de E/S) antes de poder continuar.
- **Terminado (o salida):** el proceso ha finalizado su ejecución.

18. Realice un diagrama de procesos de 5 estados.

El diagrama de estados muestra las transiciones posibles entre los cinco estados de un proceso, causadas por eventos del sistema operativo o del propio proceso:

- Nuevo $\rightarrow$ Listo: el proceso es admitido por el sistema.
- Listo $\rightarrow$ Ejecutando: el planificador despacha el proceso al procesador.
- Ejecutando $\rightarrow$ Listo: ocurre una interrupción o se agota el tiempo (quantum) asignado.
- Ejecutando $\rightarrow$ Bloqueado: el proceso solicita una operación de E/S o espera un evento.
- Bloqueado $\rightarrow$ Listo: el evento esperado ocurre.
- Ejecutando $\rightarrow$ Terminado: el proceso finaliza su ejecución.

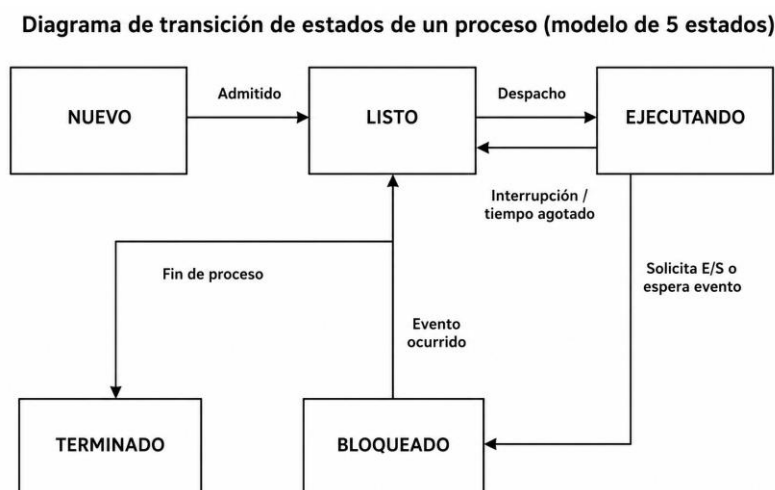


Figura 3. Diagrama de transición de estados de un proceso (modelo de 5 estados).

19. Describa el concepto de swapping.

Swapping (intercambio) es una técnica de gestión de memoria que consiste en mover temporalmente un proceso de la memoria principal (RAM) a la memoria secundaria (como un disco duro) para liberar espacio en la RAM para otros procesos.

- **Swapping out (sacar):** el sistema operativo guarda el estado completo de un proceso en disco y libera la memoria que ocupaba.
- **Swapping in (traer):** el sistema operativo recupera el proceso del disco y lo vuelve a cargar en la memoria principal.

Esto permite que el sistema tenga más procesos “listos” que la capacidad física de la RAM. Es una función de la planificación a medio plazo. Un proceso intercambiado pasa a un estado de Listo Suspendido o Bloqueado Suspendido.

20. Explique en qué consiste la memoria virtual.

La memoria virtual es una técnica de gestión de memoria que permite a un sistema operativo ejecutar programas más grandes que la memoria física (RAM) disponible. Crea la ilusión, para el usuario y los procesos, de que el sistema tiene una cantidad de memoria infinita y contigua.

Se basa en el principio de que no todo el programa necesita estar en memoria al mismo tiempo. El sistema operativo carga solo las partes (páginas o segmentos) que se están utilizando actualmente en la RAM, mientras el resto permanece en disco. Cuando se necesita una parte que no está en RAM (fallo de página), el sistema operativo la carga desde el disco, posiblemente intercambiando (swapping) otra parte fuera de la RAM. Esto permite un uso mucho más eficiente de la memoria y una mayor multiprogramación.

21. ¿En qué consiste el procesamiento distribuido?

El procesamiento distribuido se refiere al uso de múltiples sistemas de cómputo separados, interconectados por una red, para trabajar en una tarea común y coordinar sus actividades. A diferencia de un sistema multiprocesador (donde los procesadores comparten la misma memoria y un sistema operativo central), en un sistema distribuido cada nodo tiene su propia memoria y su propio sistema operativo.

La gestión de múltiples procesos que se ejecutan en estos sistemas es un área clave de estudio, donde la concurrencia se extiende a través de la red, presentando desafíos adicionales como la comunicación y la sincronización entre nodos.

22. ¿Qué es un programa multihilo?

Un programa multihilo es un programa diseñado para dividir su ejecución en varias unidades de trabajo más pequeñas llamadas hilos o threads. Todos los hilos pertenecen a un mismo proceso y comparten sus recursos (como el espacio de memoria); sin embargo, cada hilo tiene su propio estado de ejecución, contador de programa y pila.

Esto permite que un programa realice múltiples tareas de forma concurrente. Por ejemplo, en un procesador de texto, un hilo puede manejar la entrada del teclado mientras otro revisa la ortografía y un tercero guarda el archivo en segundo plano, aprovechando mejor los recursos del sistema, especialmente con múltiples núcleos.

23. En el contexto de multiprocesamiento, ¿qué es la condición de carrera?

Una condición de carrera es una situación en la que múltiples procesos o hilos acceden y manipulan datos compartidos de manera concurrente, y el resultado final de la operación depende del orden impredecible en que se ejecuten las instrucciones de cada hilo.

Es un error de sincronización. Por ejemplo, si dos hilos intentan incrementar una variable compartida (`contador++`), el resultado final puede ser incorrecto si sus operaciones de lectura, incremento y escritura se intercalan de cierta manera. Para evitarlas se deben usar mecanismos de sincronización como la exclusión mutua.

24. ¿Qué es una dirección física?

Una dirección física (también llamada dirección real o absoluta) es la dirección real de una ubicación en la memoria principal (RAM). Es el valor que el bus de direcciones del procesador utiliza para acceder a un byte de memoria.

Cuando un programa se ejecuta, el sistema operativo y el hardware (a través de la MMU) traducen las direcciones generadas por el programa (direcciones lógicas) a direcciones físicas para acceder a los datos reales en la memoria.

25. ¿Qué es una dirección lógica?

Una dirección lógica (o dirección virtual) es una dirección que genera un programa de usuario. Es la dirección vista por el proceso y el programador, y no corresponde directamente a una ubicación física en la RAM.

El sistema operativo crea para cada proceso un espacio de direcciones lógicas, dando la ilusión de que el proceso posee toda la memoria para sí mismo. La traducción a una dirección física la realiza el hardware (MMU) en conjunto con el sistema operativo.

26. ¿Qué es una dirección relativa? Dé un ejemplo.

Una dirección relativa es un tipo de dirección que se expresa como un desplazamiento respecto a una dirección base conocida, como el inicio de un programa, un segmento o un módulo.

Por ejemplo, la primera instrucción de un programa podría estar en la dirección relativa 0. Si el programa se carga en la dirección física 1000, esa instrucción tendrá la dirección física $1000 + 0 = 1000$. La siguiente instrucción, en la dirección relativa 1, tendrá la dirección física 1001. Esto permite reubicar el código en cualquier lugar de la memoria sin modificar todas las referencias, ya que se calculan sumando el desplazamiento a la dirección base.

27. Defina exclusión mutua en el contexto de ejecución de procesos.

La exclusión mutua es un requisito fundamental para la sincronización en la ejecución concurrente de procesos. Es el mecanismo que asegura que, cuando un proceso está ejecutando su sección crítica (la parte del código donde accede a recursos compartidos que no deben alterarse simultáneamente), ningún otro proceso pueda ejecutar su propia sección crítica sobre los mismos recursos.

El objetivo es garantizar que los recursos compartidos (una variable global, un archivo o una impresora) sean utilizados por un solo proceso a la vez, evitando condiciones de carrera e inconsistencias en los datos.

28. Explique espera circular en el contexto de ejecución de procesos.

La espera circular es una de las cuatro condiciones necesarias para que ocurra un interbloqueo. Se da cuando existe un conjunto de dos o más procesos donde cada uno espera un recurso retenido por otro proceso del mismo conjunto, formando un ciclo. Por ejemplo:

- El Proceso A espera por el Recurso 1, retenido por el Proceso B.
- El Proceso B espera por el Recurso 2, retenido por el Proceso A.

En este ciclo ningún proceso puede avanzar, porque todos esperan un recurso que el otro no liberará hasta terminar su ejecución, lo cual nunca sucede. Para que ocurra un interbloqueo, esta condición debe cumplirse junto con la exclusión mutua, la retención y espera, y la no apropiación.

29. ¿Qué es la sección crítica de un proceso? Dé un ejemplo.

La sección crítica de un proceso es un segmento de código en el que el proceso accede y modifica recursos compartidos (una variable global, una estructura de datos o un archivo). Es la parte del código que debe ejecutarse bajo exclusión mutua, es decir, que no puede ser ejecutada por más de un proceso o hilo al mismo tiempo.

Ejemplo: en un procedimiento almacenado que registra una persona en una base de datos, la sección crítica es el bloque de transacción SQL:

```
BEGIN TRANSACTION INSPERSONA INSERT INTO PERSONA (DNI, NOMBRES, APELLIDOS, SEXO, FECHANAC) VALUES (@DNI, @NOMBRES, @APELLIDOS, @SEXO, @FECHANAC) COMMIT TRANSACTION INSPERSONA
```

Este bloque debe ser atómico: si dos procesos lo ejecutaran al mismo tiempo, podrían intentar insertar el mismo DNI, causando un error de concurrencia.

30. Describa CPU.

La CPU (Central Processing Unit, Unidad Central de Procesamiento) es el “cerebro” del computador. Es el componente de hardware principal que ejecuta las instrucciones de los programas de software, realizando las operaciones aritméticas, lógicas y de control del sistema.

A lo largo del tiempo, las CPU han evolucionado desde chips de 4 bits como el Intel 4004 hasta arquitecturas modernas de 64 bits y múltiples núcleos (como el Core i7 de Intel). Su función central es interpretar y ejecutar un flujo de instrucciones: leer datos de la memoria (o entradas), procesarlos y escribir los resultados de vuelta en la memoria o en un dispositivo de salida.

PARCIAL 2

31. ¿En qué consiste el método de cifrado César?

El cifrado César es uno de los métodos de cifrado más antiguos y simples. Es un cifrado por sustitución en el que cada letra del texto en claro es reemplazada por otra letra que se encuentra un número fijo de posiciones más adelante en el alfabeto.

Por ejemplo, con un desplazamiento de 3 (la clave), la letra ‘A’ se convierte en ‘D’, la ‘B’ en ‘E’, y así sucesivamente. Para descifrar, se realiza el desplazamiento inverso.

32. ¿En qué consiste el método de cifrado Escítala Lacedemonia?

La Escítala Lacedemonia (o “bastón de Licurgo”) es un método de cifrado por transposición utilizado por los antiguos espartanos. No sustituye letras, sino que las reordena.

El método consiste en enrollar una tira de cuero o papiro alrededor de un bastón (la escítala). El mensaje se escribe a lo largo del bastón, de un extremo a otro, en varias líneas. Al desenrollar la tira, las letras aparecen en un orden aparentemente aleatorio. El mensaje solo puede leerse enrollando la tira en un bastón del mismo grosor.

33. Describa el cifrado por transposición.

El cifrado por transposición no altera los caracteres del mensaje, sino que los reordena o permuta siguiendo un patrón o clave definido, con el objetivo de romper la estructura del mensaje original para que no sea legible.

A diferencia del cifrado por sustitución (que cambia los caracteres), la transposición mantiene los mismos caracteres, pero en un orden diferente, como ocurre en la Escítala Lacedemonia o al escribir el mensaje en una tabla y leerlo por columnas.

34. Describa el cifrado por sustitución.

El cifrado por sustitución es una técnica en la que cada elemento del texto en claro (letras, números, etc.) es reemplazado sistemáticamente por otro elemento del alfabeto o conjunto de símbolos. El texto cifrado resultante está compuesto por los mismos tipos de elementos que el original, pero con valores diferentes.

Los ejemplos más comunes son el cifrado César (variante monoalfabética) y el cifrado polialfabético, como el de Vigenère. Es un método fundamental que se usa a menudo como parte de algoritmos criptográficos más complejos.

35. Describa el cifrado monoalfabético.

El cifrado monoalfabético es un tipo de cifrado por sustitución en el que cada letra del alfabeto del texto en claro se mapea a una letra única en el alfabeto del texto cifrado, y este mapeo (alfabeto cifrante) es fijo y se mantiene constante durante todo el mensaje.

El cifrado César es el ejemplo más simple de cifrado monoalfabético, ya que el mapeo es un desplazamiento fijo. Su principal debilidad es que es vulnerable al análisis de frecuencias de las letras en el idioma.

36. Describa el cifrado polialfabético.

El cifrado polialfabético es un tipo de cifrado por sustitución que utiliza múltiples alfabetos cifrantes a lo largo del mensaje. En lugar de un mapeo fijo como en el monoalfabético, la clave dicta qué alfabeto se usa para cifrar cada carácter, cambiando de uno a otro según un patrón.

Esto dificulta mucho el análisis de frecuencias, porque una misma letra del texto en claro puede cifrarse de diferentes maneras en distintas partes del mensaje. El ejemplo más famoso es el cifrado de Vigenère.

37. ¿En qué consiste el cifrado con clave secreta?

El cifrado con clave secreta, o criptografía simétrica, es un método que utiliza una única clave para cifrar y descifrar el mensaje. Esta clave debe compartirse en secreto entre el emisor y el receptor antes de comunicarse de forma segura.

La confidencialidad del mensaje depende enteramente de la seguridad de la clave: si un atacante la intercepta, puede leer todos los mensajes cifrados con ella. Es un método rápido y eficiente, ideal para cifrar grandes volúmenes de datos, pero su principal desafío es la distribución segura de la clave.

38. ¿En qué consiste el cifrado con clave pública?

El cifrado con clave pública, o criptografía asimétrica, utiliza un par de claves matemáticamente relacionadas: una clave pública, que puede compartirse con cualquier persona, y una clave privada, secreta y conocida solo por el propietario.

- Si se cifra un mensaje con la clave pública, solo puede descifrarse con la clave privada correspondiente.
- Si se cifra un mensaje con la clave privada, solo puede descifrarse con la clave pública.

Esto resuelve el problema de la distribución de claves, ya que la clave pública puede transmitirse abiertamente. Es fundamental para la autenticación y las firmas digitales.

39. Liste los factores de un sistema de información seguro.

- **a. Integridad:** garantiza que la información y los métodos de procesamiento sean exactos, completos y no hayan sido modificados de manera no autorizada.
- **b. Confidencialidad:** asegura que solo los usuarios autorizados puedan acceder a la información.
- **c. Disponibilidad:** garantiza que la información y los servicios estén disponibles para los usuarios autorizados cuando la necesiten.
- **d. No repudio:** proporciona una prueba irrefutable de la participación en una comunicación, impidiendo que una de las partes niegue haber realizado una acción.
 - No repudio de origen: protege al destinatario, probando que el emisor del mensaje es quien dice ser.
 - No repudio de destino: protege al emisor, evitando que el destinatario niegue haber recibido el mensaje.
- **e. Autenticación:** permite verificar la identidad de los participantes en una comunicación, asegurando el origen de la información.

40. ¿Cuáles son las propiedades más importantes de las funciones hash?

- **a. Longitud de salida fija:** independiente del tamaño del mensaje original, la huella (hash) resultante siempre tendrá el mismo tamaño (por ejemplo, 256 bits para SHA-256).
- **b. Efecto avalancha:** cualquier cambio, por pequeño que sea, en el mensaje de entrada produce un cambio masivo e impredecible en el hash de salida.
- **c. Resistencia a la primera preimagen:** dado un valor hash h , es computacionalmente inviable encontrar un mensaje m tal que $\text{hash}(m) = h$. Es unidireccional.
- **d. Resistencia a la segunda preimagen:** dado un mensaje x , es computacionalmente inviable encontrar otro mensaje x' (distinto de x) que produzca el mismo valor hash.
- **e. Resistencia a colisiones:** es computacionalmente inviable encontrar dos mensajes distintos x e y que produzcan el mismo valor hash.
- **f. Velocidad eficiente:** los algoritmos hash deben ser rápidos de calcular para ser prácticos.

41. ¿Cómo funciona un método hash?

Un método hash toma un mensaje o dato de entrada de cualquier longitud y lo procesa mediante una serie de operaciones matemáticas (desplazamientos de bits, rotaciones, operaciones lógicas no lineales) para producir una salida de longitud fija, llamada valor hash, resumen o huella digital.

El proceso es determinista (la misma entrada siempre produce el mismo hash) y está diseñado para ser unidireccional: es fácil calcular el hash a partir de la entrada, pero prácticamente imposible obtener la entrada original a partir del hash. La función se apoya en la compresión de datos y en el efecto avalancha para que incluso cambios mínimos en la entrada produzcan cambios drásticos en la salida.

42. En el contexto de los métodos hash, ¿qué significa “Resistencia a la primera preimagen”?

Significa que, dado un valor hash específico h , es computacionalmente inviable encontrar un mensaje m que, al procesarse por la función hash, produzca exactamente ese h ; es decir, no se puede invertir la función hash para descubrir la entrada original a partir de su salida.

Es la propiedad fundamental que hace que las funciones hash sean “unidireccionales”. Por ejemplo, si un sistema solo almacena el hash de una contraseña, un atacante que robe la base de datos no podrá obtener la contraseña real a partir del hash.

43. En el contexto de los métodos hash, ¿qué significa “Resistencia a la segunda preimagen”?

Significa que, dado un mensaje específico x y su hash $\text{hash}(x)$, es computacionalmente inviable encontrar otro mensaje x' , diferente de x , que produzca el mismo valor hash ($\text{hash}(x) = \text{hash}(x')$).

Esta propiedad es crucial para evitar la falsificación. Por ejemplo, si una empresa firma digitalmente un contrato x , esta resistencia impide que un atacante cree un contrato modificado x' con el mismo hash, de modo que pase la verificación de la firma digital como si fuera el original.

44. Describa lo que es un sistema embebido o empotrado.

Un sistema embebido (o empotrado) es un sistema computacional diseñado para cumplir una función específica dentro de un dispositivo más grande. A diferencia de una computadora de propósito general, su tarea está fijada de antemano y no es modificable por el usuario. Combina hardware (microcontrolador o microprocesador, memoria, sensores, actuadores) y software (firmware) para controlar el dispositivo.

Sus características principales incluyen: función dedicada, recursos limitados (poca memoria y capacidad de procesamiento), bajo consumo energético, tamaño reducido y alta confiabilidad. Un ejemplo es el sistema que controla un marcapasos o una lavadora.

45. Liste los recursos que se requieren para desarrollar un software en TinyOS.

- Un sistema operativo base (normalmente Linux o Windows).
- TinyOS (el propio sistema operativo para redes de sensores).
- nesC (el lenguaje de programación específico para TinyOS, una extensión de C).
- Entorno de desarrollo integrado (IDE) o editor de código, como Eclipse con el plugin de TinyOS.
- Compiladores de C (como gcc).
- Herramientas de programación para cargar el código en los nodos sensores (como avrdude para microcontroladores Atmel).
- Simulador (como TOSSIM) para probar el software sin hardware físico.

46. Describa la relación entre la longitud de un valor hash y la seguridad del método hash.

La relación entre la longitud de un valor hash (en bits) y su seguridad es directamente proporcional: cuanto mayor sea la longitud de la salida, mayor será la seguridad frente a ataques.

Esto se debe a que el número total de posibles valores hash es 2^n (donde n es la longitud en bits). Un atacante que quiera encontrar una colisión o invertir el hash debe probar entradas hasta dar con una que genere el mismo hash, y este espacio de búsqueda crece exponencialmente con la longitud. Por ejemplo:

- MD5 (128 bits): espacio de 2^{128} . Es vulnerable.
- SHA-1 (160 bits): espacio de 2^{160} . Hoy se considera inseguro.
- SHA-256 (256 bits): espacio de 2^{256} . Actualmente se considera seguro y es el estándar.

Además, debido al ataque de cumpleaños, la resistencia efectiva a colisiones de un hash de n bits es de aproximadamente $2^{(n/2)}$ intentos. Esto significa que, en la práctica, un hash de 256 bits ofrece una seguridad efectiva de 2^{128} frente a colisiones, un número astronómicamente grande e inviable de superar.

47. Liste 3 características de hardware que debe asignar a una máquina virtual al momento de crearla.

- **Memoria RAM:** se asigna una cantidad específica de memoria principal para que la use la máquina virtual, lo que determina cuántos programas pueden ejecutarse simultáneamente dentro de ella.
- **Núcleos de CPU (procesador):** se define el número de núcleos de la CPU anfitriona dedicados a la VM; cuantos más núcleos se asignen, mayor será su rendimiento, pero menos recursos quedarán para el sistema host.
- **Disco duro virtual (almacenamiento):** se crea un archivo en el sistema anfitrión que actúa como disco duro de la VM, definiendo su tamaño (dinámico o fijo) y formato (por ejemplo, VDI para VirtualBox, VHD para Hyper-V); el sistema operativo huésped se instala en este disco virtual.

48. Describa lo que es una colisión en un método hash.

Una colisión en un método hash ocurre cuando dos entradas o mensajes distintos producen exactamente el mismo valor hash de salida.

En teoría, las colisiones son inevitables debido al principio del palomar: existe un número infinito de mensajes posibles, pero un número finito de valores hash. La seguridad de un algoritmo hash reside en que encontrar estas colisiones sea computacionalmente inviable. Un algoritmo se considera “roto” o inseguro cuando se descubre un método práctico para generar colisiones, como ocurrió con MD5 y SHA-1.

49. ¿Cuál es el(los) requisito(s) previo(s) para instalar VirtualBox de 64 bits y crear máquinas virtuales de 64 bits en Windows 11?

- **Hardware:** un procesador de 64 bits que soporte virtualización por hardware (extensiones Intel VT-x o AMD-V).
- **BIOS/UEFI:** la función de virtualización por hardware debe estar habilitada en la configuración del BIOS/UEFI.
- **Sistema operativo:** un sistema operativo anfitrión de 64 bits, como Windows 11 de 64 bits.
- **Windows Features:** es recomendable tener desactivado Hyper-V, ya que puede entrar en conflicto con VirtualBox y ralentizar o impedir la creación de VM de 64 bits.

- **Administrador:** en Windows, el usuario debe tener permisos de administrador para instalar el software y el driver de VirtualBox.

50. ¿Los métodos hash catalogados como seguros son 100% resistentes a colisiones? Responda sí o no y explique su respuesta.

No. ningún método hash es 100% resistente a colisiones, por una razón matemática fundamental: el principio del palomar. Como el número de mensajes posibles es infinito y el número de valores hash posibles es finito (por ejemplo, 2^{256} para SHA-256), forzosamente existen múltiples mensajes que producen el mismo hash.

La seguridad de un hash radica en que, para un algoritmo como SHA-256, es computacionalmente inviable encontrar esas colisiones con la tecnología actual; la probabilidad de encontrar una por accidente es prácticamente nula y no se conoce ningún ataque eficiente que pueda lograrlo. Un algoritmo se considera “seguro” hasta que se demuestra lo contrario con un ataque práctico (como ocurrió con MD5 y SHA-1). Por lo tanto, no es 100% resistente, sino “lo suficientemente” resistente para los propósitos prácticos y estándares actuales.

51. ¿Cuál es la característica distintiva de los sistemas de tiempo real?

La característica distintiva de un sistema de tiempo real es que su corrección depende no solo del resultado lógico de la operación, sino también del momento en que ese resultado se produce. El sistema debe ser capaz de responder a los estímulos dentro de plazos (deadlines) estrictos y predecibles.

Se prioriza la previsibilidad y el determinismo sobre el rendimiento medio: una respuesta correcta que llega tarde se considera una falla.

52. ¿Cómo se gestionan los archivos en el formato FAT32?

FAT32 (File Allocation Table 32) gestiona los archivos mediante una estructura central llamada Tabla de Asignación de Archivos (FAT), que actúa como un mapa que registra, en unidades llamadas clústeres, la secuencia de bloques que ocupa cada archivo en el disco.

- Al guardar un archivo, el sistema no busca un espacio continuo; escribe los datos en los clústeres disponibles, que pueden estar dispersos (fragmentados).
- En la tabla FAT se encadenan las entradas de esos clústeres para formar una “cadena” que indica el orden de los fragmentos del archivo.
- Para leer un archivo, el sistema comienza en la primera entrada de la FAT para ese archivo, va al clúster en disco y sigue el enlace de la tabla al siguiente clúster, hasta completar el archivo.

53. ¿Cómo se gestionan los archivos en el formato NTFS?

NTFS (New Technology File System) gestiona los archivos de forma más compleja y robusta que FAT32. En lugar de una tabla FAT, utiliza una Tabla Maestra de Archivos (MFT, Master File Table).

La MFT contiene un registro para cada archivo y carpeta del volumen, con todos sus metadatos: nombre, tamaño, permisos, fechas y, críticamente, la lista de los clústeres donde se encuentran sus datos. Si un archivo es muy pequeño, sus datos pueden almacenarse directamente dentro del registro de la MFT, optimizando el espacio.

Además, NTFS implementa un sistema de registro de operaciones (journaling): antes de modificar la estructura de archivos, el sistema escribe primero en un registro (journal) lo que va a hacer. Si ocurre un fallo (corte de energía),

al reiniciar el sistema puede leer ese registro y completar o deshacer la operación para mantener la integridad del sistema de archivos.

54. ¿Cómo se gestionan los archivos en el formato Resilient File System (ReFS)?

ReFS (Resilient File System) se basa en una estructura de árbol B+ y utiliza un método llamado escritura en otro lugar (allocate-on-write) para la gestión de archivos y datos.

Cuando un archivo se modifica, ReFS no sobrescribe los datos o metadatos existentes en su ubicación original; en su lugar, escribe la nueva información en un espacio libre del disco y solo entonces actualiza la referencia (el puntero) para que apunte a la nueva ubicación. Esto reduce drásticamente el riesgo de corrupción de datos en caso de un fallo durante la escritura.

Además, ReFS aplica sumas de comprobación (checksums) a todos los metadatos y, opcionalmente, a los datos de los archivos. Al leer un archivo, el sistema verifica la suma de comprobación; si detecta que los datos no coinciden y existe una copia redundante disponible, puede corregir automáticamente la corrupción en línea, sin intervención manual, garantizando una alta resiliencia de los datos.